# Lab Report — Port Scanning & Firewall Filtering (GCP)

- **Objective:** practice network security fundamentals — service discovery via port scanning (nmap) and exposure control via firewall (VPC) rules — in an isolated lab.
- **Environment:** Google Cloud Platform. VM `lab-firewall` (Compute Engine `e2-micro`, Debian 12, zone `us-central1-a`, tag `lab-web`). Scanner: **nmap 7.94** from Cloud Shell.
- **Scope:** own lab VM only. Ephemeral resource, deleted at the end.

---

## Methodology & results

### 1. VM deployment

```
gcloud compute instances create lab-firewall --zone=us-central1-a \
  --machine-type=e2-micro --image-family=debian-12 --image-project=debian-cloud --tags=lab-web
```

### 2. Reconnaissance (full scan)

```
sudo nmap -sS -sV -sC -p- -T4 -oN recon-initial.txt <IP>
```

Result: of 65,535 ports, **only one open**.

```
Not shown: 65433 filtered tcp ports (no-response), 100 filtered (port-unreach)
22/tcp   open   ssh   OpenSSH 9.2p1 Debian 2+deb12u10
3389/tcp closed ms-wbt-server
```

Filter to open only: `grep open recon-initial.txt`

### 3. Analysis of the three observed states

| Port | State | Interpretation |
|---|---|---|
| 22 (SSH) | `open` | SSH service listening **+** GCP `default-allow-ssh` rule permits it |
| 3389 (RDP) | `closed` | `default-allow-rdp` permits it, but **no service** (Linux VM) |
| 80, 443, rest | `filtered` | **No rule** permits them → firewall drops the packet |

### 4. Control test: open a port with the firewall

```
gcloud compute firewall-rules create lab-allow-http \
  --network=default --direction=INGRESS --action=ALLOW \
  --rules=tcp:80 --target-tags=lab-web --source-ranges=0.0.0.0/0

nmap -Pn -p 80 <IP>   # Result: 80/tcp closed (was filtered)
```

The `filtered` → `closed` change confirms the firewall rule worked: the packet now **reaches the VM** (even though no service is listening).

### 5. Cleanup

```
gcloud compute firewall-rules delete lab-allow-http     # close port = delete rule
gcloud compute instances delete lab-firewall --zone=us-central1-a   # avoid cost
```

---

## Lessons & concepts applied

- **Port states:** `open` (service + allowed), `closed` (allowed, no service), `filtered` (firewall blocks). The `closed` vs `filtered` distinction reveals whether a rule permits the traffic.
- **Firewall = exposure control:** create ALLOW rule = **open**; delete rule = **close**. Apply least exposure by closing unused ports.
- **nmap only observes:** the scanner does not modify the target; the firewall opens/closes.
- **SYN scan (`-sS`):** standard half-open technique, fast and stealthy; requires privileges.
- **Recon vs verification:** first time = broad & deep (`-p-`, `-sV`, `-sC`, save output); after a change = targeted (`-Pn -p PORT`).

## Hardening finding

The VM exposed **SSH (22) to `0.0.0.0/0`** (the whole internet) via the default rule — unnecessary attack surface vulnerable to brute force. **Recommended remediation:** restrict the source to the admin's IP (`--source-ranges=YOUR_IP/32`) or use a bastion/IAP.

## Relevance & note on serverless architectures

These concepts apply to traditional servers/VMs. In **serverless** architectures (like the production site on Firebase) there are **no ports to manage**: the equivalent access control is implemented with **Firestore/Storage rules, App Check, and RBAC** (documented separately). This lab consolidates network fundamentals transferable to any traditional infrastructure.